

```
!pip install -q datasets pandas numpy scikit-learn matplotlib seaborn wordcloud langdetect
```

ERROR: Operation cancelled by user

Traceback (most recent call last):

```
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/cli/base_command.py", line 179, in exc_logging_wrapper
    status = run_func(*args)
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/cli/req_command.py", line 67, in wrapper
    return func(self, options, args)
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/commands/install.py", line 377, in run
    requirement_set = resolver.resolve(
        reqs, check_supported_wheels=not options.target_dir
    )
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/resolution/resolvelib/resolver.py", line 95, in resolve
    result = self._result = resolver.resolve(
        ~~~~~^
        collected.requirements, max_rounds=limit_how_complex_resolution_can_be
        ~~~~~^
    )
    ^
File "/usr/local/lib/python3.13/dist-packages/pip/_vendor/resolvelib/resolvers.py", line 546, in resolve
    state = resolution.resolve(requirements, max_rounds=max_rounds)
File "/usr/local/lib/python3.13/dist-packages/pip/_vendor/resolvelib/resolvers.py", line 427, in resolve
    failure_causes = self._attempt_to_pin_criterion(name)
File "/usr/local/lib/python3.13/dist-packages/pip/_vendor/resolvelib/resolvers.py", line 239, in _attempt_to_pin_criterion
    criteria = self._get_updated_criteria(candidate)
File "/usr/local/lib/python3.13/dist-packages/pip/_vendor/resolvelib/resolvers.py", line 229, in _get_updated_criteria
    for requirement in self._p.get_dependencies(candidate=candidate):
        ~~~~~^
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/resolution/resolvelib/provider.py", line 247, in get_dependencies
    return [r for r in candidate.iter_dependencies(with_requires) if r is not None]
        ~~~~~^
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/resolution/resolvelib/candidates.py", line 518, in iter_dependencies
    for r in self.base.dist.iter_dependencies(valid_extras):
        ~~~~~^
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/metadata/importlib/_dists.py", line 227, in iter_dependencies
    elif any(req.marker.evaluate(context) for context in contexts):
        ~~~~~^
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/metadata/importlib/_dists.py", line 227, in <genexpr>
    elif any(req.marker.evaluate(context) for context in contexts):
        ~~~~~^
File "/usr/local/lib/python3.13/dist-packages/pip/_vendor/packaging/markers.py", line 310, in evaluate
    current_environment = cast("dict[str, str]", default_environment())
        ~~~~~^
File "/usr/local/lib/python3.13/dist-packages/pip/_vendor/packaging/markers.py", line 243, in default_environment
    iver = format_full_version(sys.implementation.version)
File "/usr/local/lib/python3.13/dist-packages/pip/_vendor/packaging/markers.py", line 235, in format_full_version
    version = "{0.major}.{0.minor}.{0.micro}".format(info)
```

KeyboardInterrupt

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "/usr/local/bin/pip3", line 10, in <module>
    sys.exit(main())
    ~~~~~^
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/cli/main.py", line 80, in main
    return command.main(cmd_args)
    ~~~~~^
File "/usr/local/lib/python3.13/dist-packages/pip/_internal/cli/base_command.py", line 100, in main
    return self._main(args)
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings

warnings.filterwarnings("ignore")

from datasets import load_dataset

from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    cross_validate
)

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
```

```
classification_report,  
confusion_matrix,  
roc_auc_score  
)  
  
print("Libraries imported successfully! ✅")
```

Libraries imported successfully! ✅

```
dataset = load_dataset(  
    "Flowerly/modern-fake-reviews"  
)  
  
print(dataset)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster download  
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable hi  
README.md: 100% 3.62k/3.62k [00:00<00:00, 274kB/s]  
  
data/train-00000-of-00001.parquet: reconstructing file: 100% 7.40MB / 7.40MB, 730kB/s  
data/train-00000-of-00001.parquet: downloading bytes: 7.39MB, 727kB/s  
data/validation-00000-of-00001.parquet: reconstructing file: 100% 529kB / 529kB, 52.6kB/s  
data/validation-00000-of-00001.parquet: downloading bytes: 527kB, 52.4kB/s  
data/test-00000-of-00001.parquet: reconstructing file: 100% 1.28MB / 1.28MB, 127kB/s  
data/test-00000-of-00001.parquet: downloading bytes: 1.28MB, 127kB/s  
  
Generating train split: 100% 32339/32339 [00:00<00:00, 215251.41 examples/s]  
Generating validation split: 100% 2425/2425 [00:00<00:00, 82592.53 examples/s]  
Generating test split: 100% 5660/5660 [00:00<00:00, 115173.35 examples/s]  
DatasetDict({  
  train: Dataset({  
    features: ['category', 'rating', 'label', 'text_'],  
    num_rows: 32339  
  })  
  validation: Dataset({  
    features: ['category', 'rating', 'label', 'text_'],  
    num_rows: 2425  
  })  
  test: Dataset({  
    features: ['category', 'rating', 'label', 'text_'],  
    num_rows: 5660  
  })  
})
```

```
print(dataset.keys())  
  
dict_keys(['train', 'validation', 'test'])
```

```
for split in dataset.keys():  
    print(  
        split,  
        "→",  
        dataset[split].num_rows,  
        "rows"  
    )
```

train → 32339 rows
validation → 2425 rows
test → 5660 rows

```
df = dataset["train"].to_pandas()  
  
print("Dataset shape:", df.shape)  
  
df.head()
```

Dataset shape: (32339, 4)

	category	rating	label	text_
0	Pet_Supplies_5	3.0	CG	Maybe okay for a calm small dog, but not for m...
1	Pet_Supplies_5	5.0	CG	Order arrived fast and in perfect condition, n...
2	Movies_and_TV_5	5.0	CG	Went in with sky-high hopes and it still blew ...
3	Clothing_Shoes_and_Jewelry_5	3.0	CG	These pants fit my tall, lanky frame better th...
4	Clothing_Shoes_and_Jewelry_5	5.0	CG	I've had these for a few months now and honest...

```
print(df.columns.tolist())
```

```
['category', 'rating', 'label', 'text_']
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32339 entries, 0 to 32338
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   category    32339 non-null  object
 1   rating      32339 non-null  float64
 2   label       32339 non-null  object
 3   text_       32339 non-null  object
dtypes: float64(1), object(3)
memory usage: 1010.7+ KB
```

```
print("Label Distribution:")
print(df["label"].value_counts())
```

```
Label Distribution:
label
CG    16172
OR    16167
Name: count, dtype: int64
```

```
print("\nLabel Percentage:")
print(
    df["label"]
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)
```

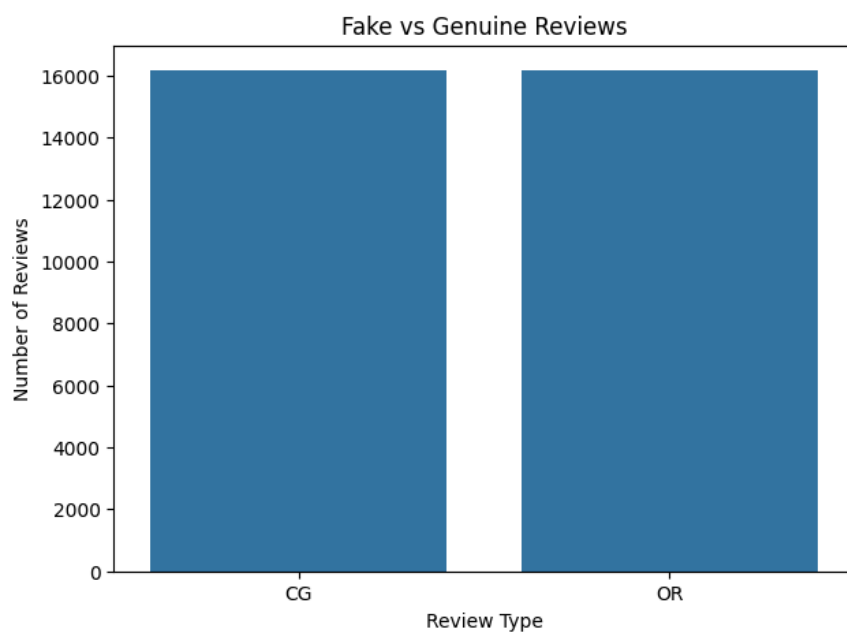
```
Label Percentage:
label
CG    50.01
OR    49.99
Name: proportion, dtype: float64
```

```
plt.figure(figsize=(7, 5))

sns.countplot(data=df, x="label")

plt.title("Fake vs Genuine Reviews")
plt.xlabel("Review Type")
plt.ylabel("Number of Reviews")

plt.show()
```



```
print("Missing Values in Each Column:")
print(df.isnull().sum())
```

```
Missing Values in Each Column:
category    0
rating      0
label       0
text_       0
dtype: int64
```

```
duplicate_count = df["text_"].duplicated().sum()
print("Duplicate Reviews:", duplicate_count)
```

Duplicate Reviews: 1

```
print("Dataset Shape:", df.shape)
print("\nData Types:")
print(df.dtypes)
```

Dataset Shape: (32339, 4)

Data Types:
category object
rating float64
label object
text_ object
dtype: object

```
display(df.head())
```

	category	rating	label	text_
0	Pet_Supplies_5	3.0	CG	Maybe okay for a calm small dog, but not for m...
1	Pet_Supplies_5	5.0	CG	Order arrived fast and in perfect condition, n...
2	Movies_and_TV_5	5.0	CG	Went in with sky-high hopes and it still blew ...
3	Clothing_Shoes_and_Jewelry_5	3.0	CG	These pants fit my tall, lanky frame better th...
4	Clothing_Shoes_and_Jewelry_5	5.0	CG	I've had these for a few months now and honest...

```
import re

def clean_text(text):
    text = str(text).lower()

    # Remove URLs
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)

    # Remove HTML tags
    text = re.sub(r"<.*?>", "", text)

    # Remove extra spaces
    text = re.sub(r"\s+", " ", text)

    return text.strip()

# Apply cleaning
df["clean_text"] = df["text_"].apply(clean_text)

# Show original vs cleaned text
display(df[["text_", "clean_text", "label"]].head())
```

	text_	clean_text	label
0	Maybe okay for a calm small dog, but not for m...	maybe okay for a calm small dog, but not for m...	CG
1	Order arrived fast and in perfect condition, n...	order arrived fast and in perfect condition, n...	CG
2	Went in with sky-high hopes and it still blew ...	went in with sky-high hopes and it still blew ...	CG
3	These pants fit my tall, lanky frame better th...	these pants fit my tall, lanky frame better th...	CG
4	I've had these for a few months now and honest...	i've had these for a few months now and honest...	CG

```
print("Original Review:")
print(df["text_"].iloc[0])

print("\nCleaned Review:")
print(df["clean_text"].iloc[0])
```

Original Review:
Maybe okay for a calm small dog, but not for my stubborn beagle.

Cleaned Review:
maybe okay for a calm small dog, but not for my stubborn beagle.

```
empty_reviews = (df["clean_text"].str.len() == 0).sum()

print("Empty Reviews:", empty_reviews)
```

Empty Reviews: 0

```
print("Rating Distribution:")
print(df["rating"].value_counts().sort_index())
```

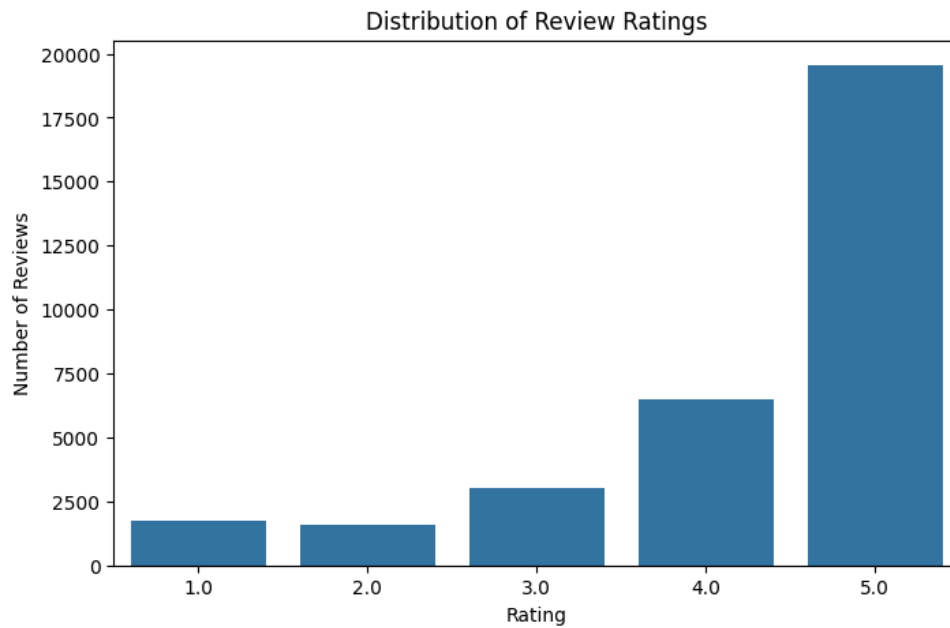
```
Rating Distribution:
rating
1.0    1725
2.0    1600
3.0    2999
4.0    6470
5.0   19545
Name: count, dtype: int64
```

```
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="rating")

plt.title("Distribution of Review Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Reviews")

plt.show()
```



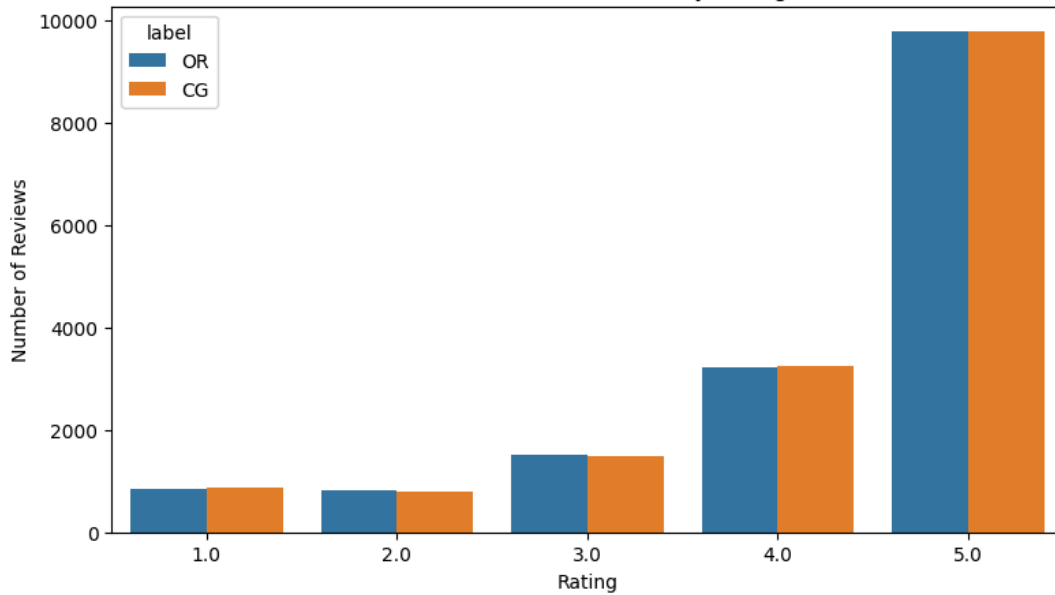
```
plt.figure(figsize=(9, 5))

sns.countplot(
    data=df,
    x="rating",
    hue="label"
)

plt.title("Fake vs Genuine Reviews by Rating")
plt.xlabel("Rating")
plt.ylabel("Number of Reviews")

plt.show()
```

Fake vs Genuine Reviews by Rating



```
df["review_length"] = df["text_"].astype(str).apply(len)

print(df[["text_", "review_length", "label"]].head())
```

	text_	review_length	label
0	Maybe okay for a calm small dog, but not for m...	64	CG
1	Order arrived fast and in perfect condition, n...	281	CG
2	Went in with sky-high hopes and it still blew ...	54	CG
3	These pants fit my tall, lanky frame better th...	354	CG
4	I've had these for a few months now and honest...	693	CG

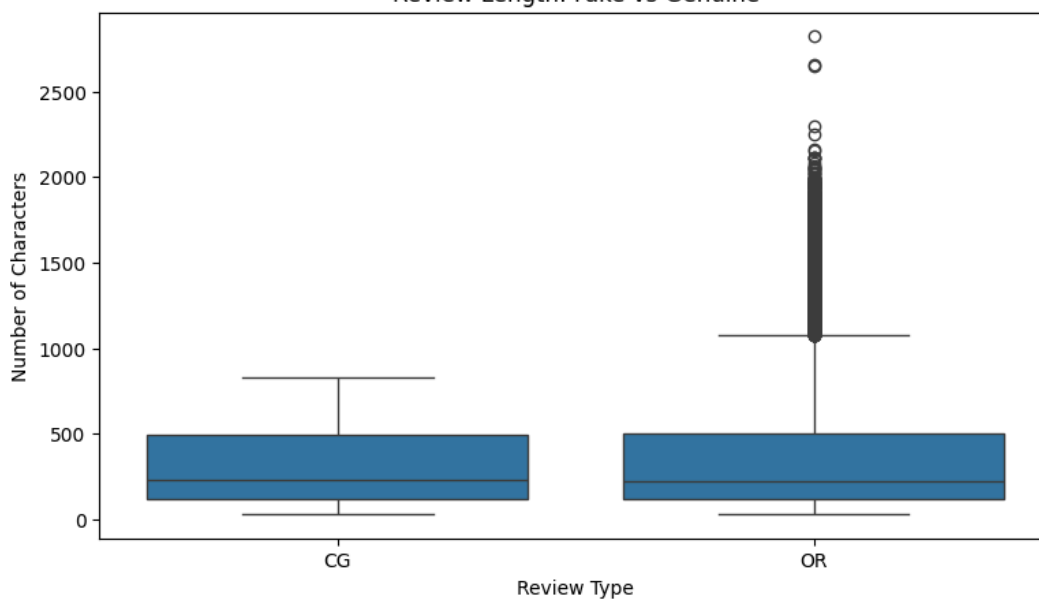
```
plt.figure(figsize=(9, 5))

sns.boxplot(
    data=df,
    x="label",
    y="review_length"
)

plt.title("Review Length: Fake vs Genuine")
plt.xlabel("Review Type")
plt.ylabel("Number of Characters")

plt.show()
```

Review Length: Fake vs Genuine



```
from sklearn.model_selection import train_test_split

X = df["clean_text"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
```

```

X,
y,
test_size=0.20,
random_state=42,
stratify=y
)

print("Training Reviews:", len(X_train))
print("Testing Reviews:", len(X_test))

print("\nTraining Label Distribution:")
print(y_train.value_counts())

print("\nTesting Label Distribution:")
print(y_test.value_counts())

```

```

Training Reviews: 25871
Testing Reviews: 6468

Training Label Distribution:
label
CG    12937
OR    12934
Name: count, dtype: int64

Testing Label Distribution:
label
CG     3235
OR     3233
Name: count, dtype: int64

```

```

print("Total Reviews:", len(df))
print("Train + Test:", len(X_train) + len(X_test))

```

```

Total Reviews: 32339
Train + Test: 32339

```

```

from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    max_features=50000,
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.95,
    sublinear_tf=True
)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("Training TF-IDF Shape:", X_train_tfidf.shape)
print("Testing TF-IDF Shape:", X_test_tfidf.shape)

```

```

Training TF-IDF Shape: (25871, 50000)
Testing TF-IDF Shape: (6468, 50000)

```

```

from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    random_state=42
)

model.fit(X_train_tfidf, y_train)

print("Model training completed successfully! ✅")

```

```

Model training completed successfully! ✅

```

```

y_pred = model.predict(X_test_tfidf)

print("Predictions completed! ✅")
print("\nFirst 20 Predictions:")
print(y_pred[:20])

```

```

Predictions completed! ✅

First 20 Predictions:
['CG' 'OR' 'OR' 'CG' 'CG' 'OR' 'OR' 'OR' 'OR' 'CG' 'CG' 'OR' 'CG' 'OR'
 'OR' 'OR' 'OR' 'CG' 'OR' 'CG']

```

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,

```

```

        f1_score
    )

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test, y_pred, pos_label="CG"
    )
    recall = recall_score(
        y_test, y_pred, pos_label="CG"
    )
    f1 = f1_score(
        y_test, y_pred, pos_label="CG"
    )

    print("Model Performance")
    print("-----")
    print(f"Accuracy : {accuracy:.4f}")
    print(f"Precision : {precision:.4f}")
    print(f"Recall : {recall:.4f}")
    print(f"F1-Score : {f1:.4f}")

```

```

Model Performance
-----
Accuracy : 0.9839
Precision : 0.9810
Recall : 0.9870
F1-Score : 0.9840

```

```

from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Fake (CG)", "Genuine (OR)"]
    )
)

```

	precision	recall	f1-score	support
Fake (CG)	0.98	0.99	0.98	3235
Genuine (OR)	0.99	0.98	0.98	3233
accuracy			0.98	6468
macro avg	0.98	0.98	0.98	6468
weighted avg	0.98	0.98	0.98	6468

```

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(
    y_test,
    y_pred,
    labels=["CG", "OR"]
)

plt.figure(figsize=(7, 5))

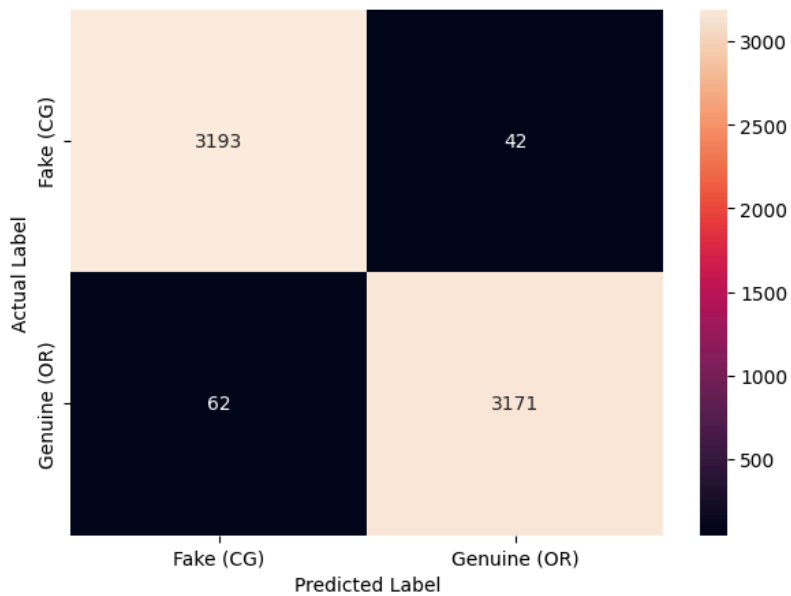
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=["Fake (CG)", "Genuine (OR)"],
    yticklabels=["Fake (CG)", "Genuine (OR)"]
)

plt.title("Confusion Matrix – Fake Review Detection")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.show()

```

Confusion Matrix — Fake Review Detection



```
from sklearn.metrics import roc_auc_score

# Probability of Fake (CG)
fake_index = list(model.classes_).index("CG")

y_prob = model.predict_proba(X_test_tfidf)[:, fake_index]

# Convert actual labels to binary
y_test_binary = (y_test == "CG").astype(int)

roc_auc = roc_auc_score(
    y_test_binary,
    y_prob
)

print(f"ROC-AUC Score: {roc_auc:.4f}")
```

ROC-AUC Score: 0.9985

```
print("Total Categories:", df["category"].nunique())

print("\nCategories:")
print(df["category"].unique())
```

Total Categories: 10

```
Categories:
['Pet_Supplies_5' 'Movies_and_TV_5' 'Clothing_Shoes_and_Jewelry_5'
 'Home_and_Kitchen_5' 'Tools_and_Home_Improvement_5' 'Toys_and_Games_5'
 'Books_5' 'Sports_and_Outdoors_5' 'Electronics_5' 'Kindle_Store_5']
```

```
category_counts = df["category"].value_counts()

print(category_counts)
```

```
category
Kindle_Store_5      3793
Books_5             3489
Pet_Supplies_5      3405
Home_and_Kitchen_5  3293
Electronics_5       3201
Sports_and_Outdoors_5  3136
Tools_and_Home_Improvement_5  3067
Clothing_Shoes_and_Jewelry_5  3041
Toys_and_Games_5    3027
Movies_and_TV_5     2887
Name: count, dtype: int64
```

```
category_label = pd.crosstab(
    df["category"],
    df["label"]
)

display(category_label)
```

label	CG	OR
category		
Books_5	1754	1735
Clothing_Shoes_and_Jewelry_5	1511	1530
Electronics_5	1601	1600
Home_and_Kitchen_5	1645	1648
Kindle_Store_5	1897	1896
Movies_and_TV_5	1440	1447
Pet_Supplies_5	1711	1694
Sports_and_Outdoors_5	1573	1563
Tools_and_Home_Improvement_5	1518	1549
Toys_and_Games_5	1522	1505

```
print("Number of categories:", df["category"].nunique())
```

```
print("\nMinimum reviews in a category:")
print(df["category"].value_counts().min())
```

Number of categories: 10

Minimum reviews in a category:
2887

```
categories = df["category"].unique()
```

```
np.random.seed(42)
np.random.shuffle(categories)
```

```
split_point = int(len(categories) * 0.80)
```

```
train_categories = categories[:split_point]
test_categories = categories[split_point:]
```

```
print("Training Categories:", len(train_categories))
print("Testing Categories:", len(test_categories))
```

```
print("\nTest Categories:")
print(test_categories)
```

Training Categories: 8
Testing Categories: 2

Test Categories:
['Home_and_Kitchen_5' 'Books_5']

```
train_df = df[df["category"].isin(train_categories)].copy()
test_df = df[df["category"].isin(test_categories)].copy()
```

```
print("Training Reviews:", len(train_df))
print("Testing Reviews:", len(test_df))
```

```
print("\nCategory Overlap:")
print(
    set(train_df["category"]).intersection(
        set(test_df["category"])
    )
)
```

Training Reviews: 25557
Testing Reviews: 6782

Category Overlap:
set()

```
X_train_cat = train_df["clean_text"]
y_train_cat = train_df["label"]
```

```
X_test_cat = test_df["clean_text"]
y_test_cat = test_df["label"]
```

```
tfidf_cat = TfidfVectorizer(
    max_features=50000,
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.95,
    sublinear_tf=True
)
```

```
X_train_cat_tfidf = tfidf_cat.fit_transform(X_train_cat)
X_test_cat_tfidf = tfidf_cat.transform(X_test_cat)

print("Training TF-IDF:", X_train_cat_tfidf.shape)
print("Testing TF-IDF:", X_test_cat_tfidf.shape)
```

Training TF-IDF: (25557, 50000)
Testing TF-IDF: (6782, 50000)

```
category_model = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    random_state=42
)

category_model.fit(
    X_train_cat_tfidf,
    y_train_cat
)

y_pred_cat = category_model.predict(
    X_test_cat_tfidf
)

print("Category-held-out model trained successfully! ✅")
```

Category-held-out model trained successfully! ✅

```
accuracy_cat = accuracy_score(y_test_cat, y_pred_cat)

precision_cat = precision_score(
    y_test_cat,
    y_pred_cat,
    pos_label="CG"
)

recall_cat = recall_score(
    y_test_cat,
    y_pred_cat,
    pos_label="CG"
)

f1_cat = f1_score(
    y_test_cat,
    y_pred_cat,
    pos_label="CG"
)

print("CATEGORY-HELD-OUT RESULTS")
print("-----")
print(f"Accuracy : {accuracy_cat:.4f}")
print(f"Precision : {precision_cat:.4f}")
print(f"Recall : {recall_cat:.4f}")
print(f"F1-Score : {f1_cat:.4f}")
```

CATEGORY-HELD-OUT RESULTS

Accuracy : 0.9851
Precision : 0.9884
Recall : 0.9818
F1-Score : 0.9851

```
print(
    classification_report(
        y_test_cat,
        y_pred_cat,
        target_names=["Fake (CG)", "Genuine (OR)"]
    )
)
```

	precision	recall	f1-score	support
Fake (CG)	0.99	0.98	0.99	3399
Genuine (OR)	0.98	0.99	0.99	3383
accuracy			0.99	6782
macro avg	0.99	0.99	0.99	6782
weighted avg	0.99	0.99	0.99	6782

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_validate

cv_pipeline = Pipeline([
    (
```

```

        "tfidf",
        TfidfVectorizer(
            max_features=50000,
            ngram_range=(1, 2),
            min_df=2,
            max_df=0.95,
            sublinear_tf=True
        )
    ),
    (
        "model",
        LogisticRegression(
            max_iter=1000,
            class_weight="balanced",
            random_state=42
        )
    )
])

print("Pipeline created successfully! ✅")

```

Pipeline created successfully! ✅

```

X_cv = df["clean_text"]
y_cv = df["label"]

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

scoring = {
    "accuracy": "accuracy",
    "precision": "precision_macro",
    "recall": "recall_macro",
    "f1": "f1_macro"
}

cv_results = cross_validate(
    cv_pipeline,
    X_cv,
    y_cv,
    cv=cv,
    scoring=scoring,
    n_jobs=-1
)

print("5-Fold Cross-Validation Completed! ✅")

```

5-Fold Cross-Validation Completed! ✅

```

print("5-FOLD CROSS-VALIDATION RESULTS")
print("-----")

print(
    f"Accuracy : "
    f"{cv_results['test_accuracy'].mean():.4f} "
    f"+/- {cv_results['test_accuracy'].std():.4f}"
)

print(
    f"Precision: "
    f"{cv_results['test_precision'].mean():.4f} "
    f"+/- {cv_results['test_precision'].std():.4f}"
)

print(
    f"Recall   : "
    f"{cv_results['test_recall'].mean():.4f} "
    f"+/- {cv_results['test_recall'].std():.4f}"
)

print(
    f"F1-Score : "
    f"{cv_results['test_f1'].mean():.4f} "
    f"+/- {cv_results['test_f1'].std():.4f}"
)

```

```

5-FOLD CROSS-VALIDATION RESULTS
-----
Accuracy : 0.9843 +/- 0.0008
Precision : 0.9843 +/- 0.0008
Recall    : 0.9843 +/- 0.0008
F1-Score  : 0.9843 +/- 0.0008

```

```

cv_table = pd.DataFrame({
    "Fold": [1, 2, 3, 4, 5],
    "Accuracy": cv_results["test_accuracy"],
    "Precision": cv_results["test_precision"],
    "Recall": cv_results["test_recall"],
    "F1-Score": cv_results["test_f1"]
})

display(cv_table.round(4))

```

	Fold	Accuracy	Precision	Recall	F1-Score
0	1	0.9848	0.9849	0.9848	0.9848
1	2	0.9848	0.9848	0.9848	0.9848
2	3	0.9844	0.9844	0.9844	0.9844
3	4	0.9827	0.9827	0.9827	0.9827
4	5	0.9848	0.9848	0.9848	0.9848

```
!pip install -q datasets
```

```

import datasets
import huggingface_hub

print("datasets version:", datasets.__version__)
print("huggingface_hub version:", huggingface_hub.__version__)

```

```

datasets version: 4.8.5
huggingface_hub version: 1.29.0

```

```

def predict_review(review):
    cleaned = clean_text(review)

    # Convert review to TF-IDF
    review_tfidf = tfidf.transform([cleaned])

    # Prediction
    prediction = model.predict(review_tfidf)[0]

    # Probability
    probabilities = model.predict_proba(review_tfidf)[0]

    fake_index = list(model.classes_).index("CG")
    fake_probability = probabilities[fake_index] * 100

    if prediction == "CG":
        result = "FAKE REVIEW"
    else:
        result = "GENUINE REVIEW"

    print("Review:")
    print(review)
    print("\nPrediction:", result)
    print(f"Fake Probability: {fake_probability:.2f}%")

    return prediction

```

```

review = """
The product arrived in perfect condition.
The quality is excellent and I am very happy
with my purchase.
"""

predict_review(review)

```

Review:

The product arrived in perfect condition.
The quality is excellent and I am very happy
with my purchase.

Prediction: GENUINE REVIEW
Fake Probability: 8.31%
'OR'

```

review = """
Amazing product!!! Best ever!!! Everyone should
buy this immediately!!! Five stars!!!
"""

predict_review(review)

```

Review:

Amazing product!!! Best ever!!! Everyone should
buy this immediately!!! Five stars!!!

Prediction: GENUINE REVIEW

Fake Probability: 6.28%

'OR'

```
product_stats = df.groupby("category").agg(
    total_reviews=("text_", "count"),
    average_rating=("rating", "mean"),
    fake_reviews=("label", lambda x: (x == "CG").sum()),
    genuine_reviews=("label", lambda x: (x == "OR").sum())
).reset_index()

product_stats["fake_percentage"] = (
    product_stats["fake_reviews"] /
    product_stats["total_reviews"] * 100
)

product_stats["genuine_percentage"] = (
    product_stats["genuine_reviews"] /
    product_stats["total_reviews"] * 100
)

product_stats["average_rating"] = (
    product_stats["average_rating"].round(2)
)

display(product_stats.head())
```

	category	total_reviews	average_rating	fake_reviews	genuine_reviews	fake_percentage	genuine_percentage
0	Books_5	3489	4.28	1754	1735	50.272284	49.727716
1	Clothing_Shoes_and_Jewelry_5	3041	4.16	1511	1530	49.687603	50.312397
2	Electronics_5	3201	4.16	1601	1600	50.015620	49.984380
3	Home_and_Kitchen_5	3293	4.24	1645	1648	49.954449	50.045551
4	Kindle_Store_5	3793	4.39	1897	1896	50.013182	49.986818

```
print("Number of categories:", len(product_stats))
print("Columns:")
print(product_stats.columns.tolist())

print("\nShape:", product_stats.shape)
```

```
Number of categories: 10
Columns:
['category', 'total_reviews', 'average_rating', 'fake_reviews', 'genuine_reviews', 'fake_percentage', 'genuine_percentage']

Shape: (10, 7)
```

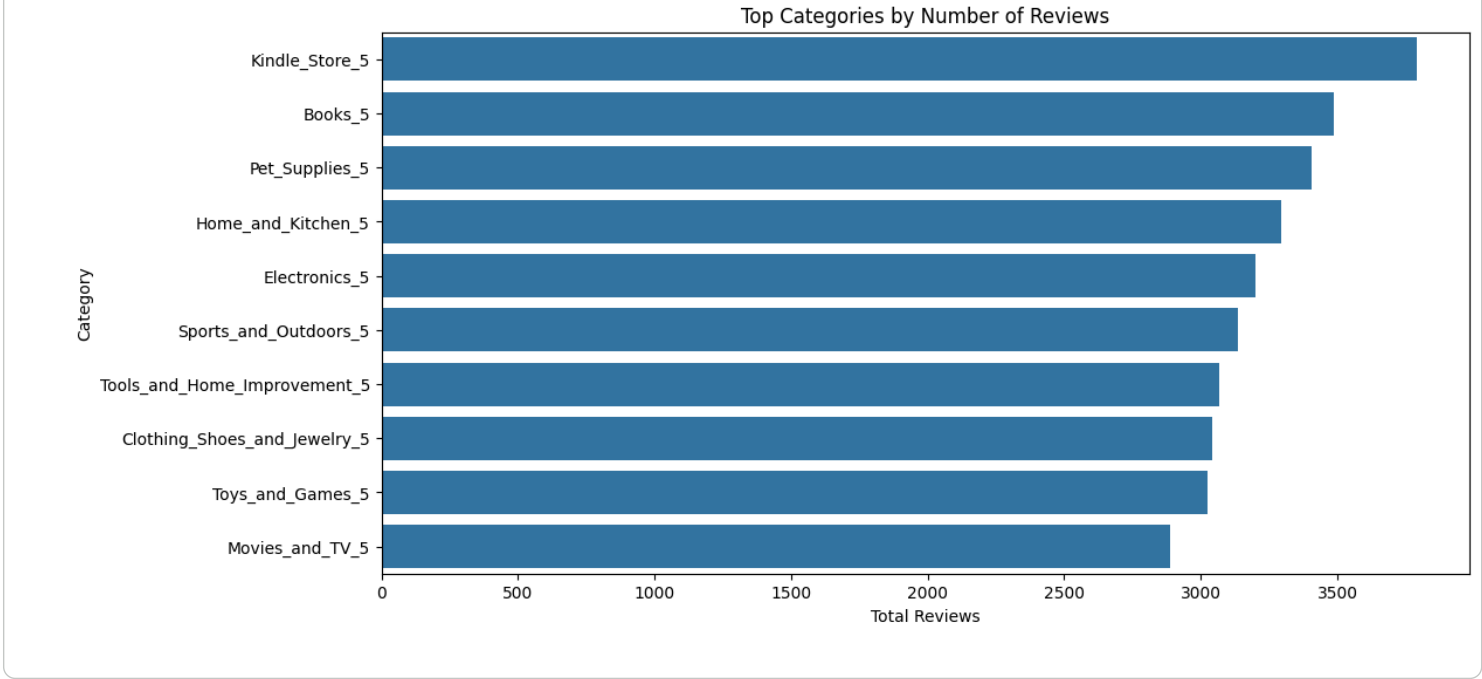
```
plt.figure(figsize=(12, 6))

top_categories = (
    product_stats
    .sort_values("total_reviews", ascending=False)
    .head(15)
)

sns.barplot(
    data=top_categories,
    x="total_reviews",
    y="category"
)

plt.title("Top Categories by Number of Reviews")
plt.xlabel("Total Reviews")
plt.ylabel("Category")

plt.show()
```



```
# Estimated Product Views
# NOTE: These are simulated estimates, NOT real Amazon traffic data.

np.random.seed(42)

product_stats["estimated_views"] = (
    product_stats["total_reviews"] * 30
    + product_stats["average_rating"] * 500
    + product_stats["genuine_percentage"] * 100
    + np.random.normal(
        0,
        1000,
        len(product_stats)
    )
)

product_stats["estimated_views"] = (
    product_stats["estimated_views"]
    .clip(lower=100)
    .round()
    .astype(int)
)

display(
    product_stats[
        [
            "category",
            "total_reviews",
            "average_rating",
            "fake_percentage",
            "genuine_percentage",
            "estimated_views"
        ]
    ].head(10)
)
```

	category	total_reviews	average_rating	fake_percentage	genuine_percentage	estimated_views
0	Books_5	3489	4.28	50.272284	49.727716	112279
1	Clothing_Shoes_and_Jewelry_5	3041	4.16	49.687603	50.312397	98203
2	Electronics_5	3201	4.16	50.015620	49.984380	103756
3	Home_and_Kitchen_5	3293	4.24	49.954449	50.045551	107438
4	Kindle_Store_5	3793	4.39	50.013182	49.986818	120750
5	Movies_and_TV_5	2887	4.10	49.878767	50.121233	93438
6	Pet_Supplies_5	3405	4.21	50.249633	49.750367	110809
7	Sports_and_Outdoors_5	3136	4.29	50.159439	49.840561	101976
8	Tools_and_Home_Improvement_5	3067	4.32	49.494620	50.505380	98751
9	Toys_and_Games_5	3027	4.34	50.280806	49.719194	98494

```
print(
    product_stats[
        [
```

```

        "total_reviews",
        "average_rating",
        "fake_percentage",
        "genuine_percentage",
        "estimated_views"
    ]
].corr()["estimated_views"].sort_values(ascending=False)
)

```

```

estimated_views      1.000000
total_reviews        0.996516
average_rating        0.511904
fake_percentage       0.408173
genuine_percentage   -0.408173
Name: estimated_views, dtype: float64

```

```

features = [
    "total_reviews",
    "average_rating",
    "fake_percentage",
    "genuine_percentage"
]

X_views = product_stats[features]
y_views = product_stats["estimated_views"]

print("Features:")
print(features)

print("\nX Shape:", X_views.shape)
print("Y Shape:", y_views.shape)

```

```

Features:
['total_reviews', 'average_rating', 'fake_percentage', 'genuine_percentage']

X Shape: (10, 4)
Y Shape: (10,)

```

```

from sklearn.model_selection import train_test_split

X_train_views, X_test_views, y_train_views, y_test_views = train_test_split(
    X_views,
    y_views,
    test_size=0.20,
    random_state=42
)

print("Training samples:", len(X_train_views))
print("Testing samples:", len(X_test_views))

```

```

Training samples: 8
Testing samples: 2

```

```

from sklearn.ensemble import RandomForestRegressor

rf_views = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

rf_views.fit(
    X_train_views,
    y_train_views
)

views_pred = rf_views.predict(X_test_views)

print("Random Forest Views Model trained successfully! ✅")

```

```

Random Forest Views Model trained successfully! ✅

```

```

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae = mean_absolute_error(
    y_test_views,
    views_pred
)

rmse = np.sqrt(
    mean_squared_error(
        y_test_views,
        views_pred
    )
)

```

```

    )

r2 = r2_score(
    y_test_views,
    views_pred
)

print("PRODUCT VIEWS PREDICTION RESULTS")
print("-----")
print(f"MAE : {mae:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R² : {r2:.4f}")

```

```

PRODUCT VIEWS PREDICTION RESULTS
-----
MAE : 911.18
RMSE : 916.47
R² : -10.1876

```

```

comparison = pd.DataFrame({
    "Actual Views": y_test_views.values,
    "Predicted Views": views_pred.round().astype(int)
})

display(comparison.head(10))

```

	Actual Views	Predicted Views
0	98751	99760
1	98203	99016

```

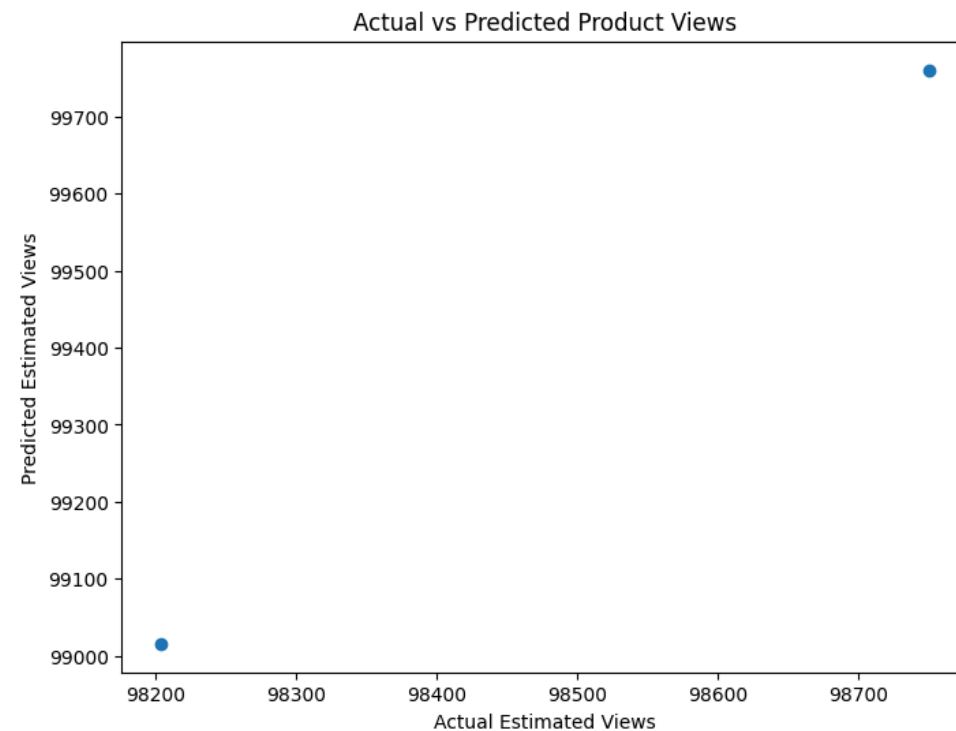
plt.figure(figsize=(8, 6))

plt.scatter(
    y_test_views,
    views_pred
)

plt.xlabel("Actual Estimated Views")
plt.ylabel("Predicted Estimated Views")
plt.title("Actual vs Predicted Product Views")

plt.show()

```



```

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(
        n_estimators=200,
        random_state=42
    )
}

```

```

    ),
    "Gradient Boosting": GradientBoostingRegressor(
        n_estimators=200,
        random_state=42
    )
}

results = []

for name, model in models.items():
    model.fit(X_train_views, y_train_views)

    predictions = model.predict(X_test_views)

    mae = mean_absolute_error(
        y_test_views,
        predictions
    )

    rmse = np.sqrt(
        mean_squared_error(
            y_test_views,
            predictions
        )
    )

    r2 = r2_score(
        y_test_views,
        predictions
    )

    results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2
    })

print("All models trained successfully! ✅")

```

All models trained successfully! ✅

```

results_df = pd.DataFrame(results)

results_df = results_df.sort_values(
    "R2",
    ascending=False
).reset_index(drop=True)

display(
    results_df.round(4)
)

```

	Model	MAE	RMSE	R2
0	Linear Regression	293.8377	378.7923	-0.9112
1	Random Forest	911.1850	916.4715	-10.1876
2	Gradient Boosting	1551.8354	1556.2607	-31.2599

```

plt.figure(figsize=(9, 5))

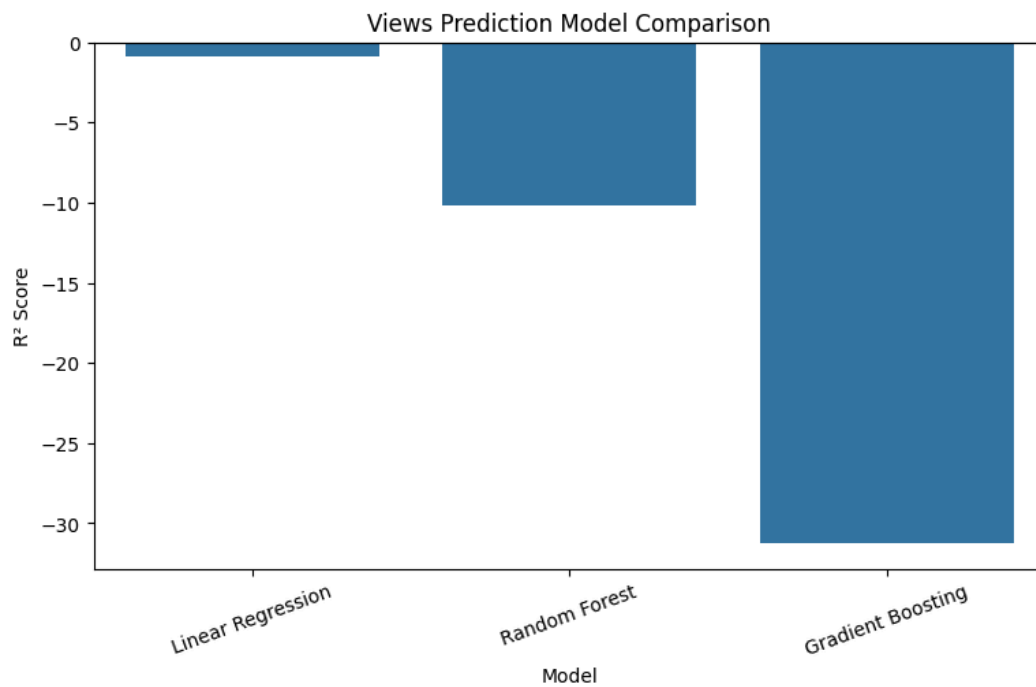
sns.barplot(
    data=results_df,
    x="Model",
    y="R2"
)

plt.title("Views Prediction Model Comparison")
plt.xlabel("Model")
plt.ylabel("R2 Score")

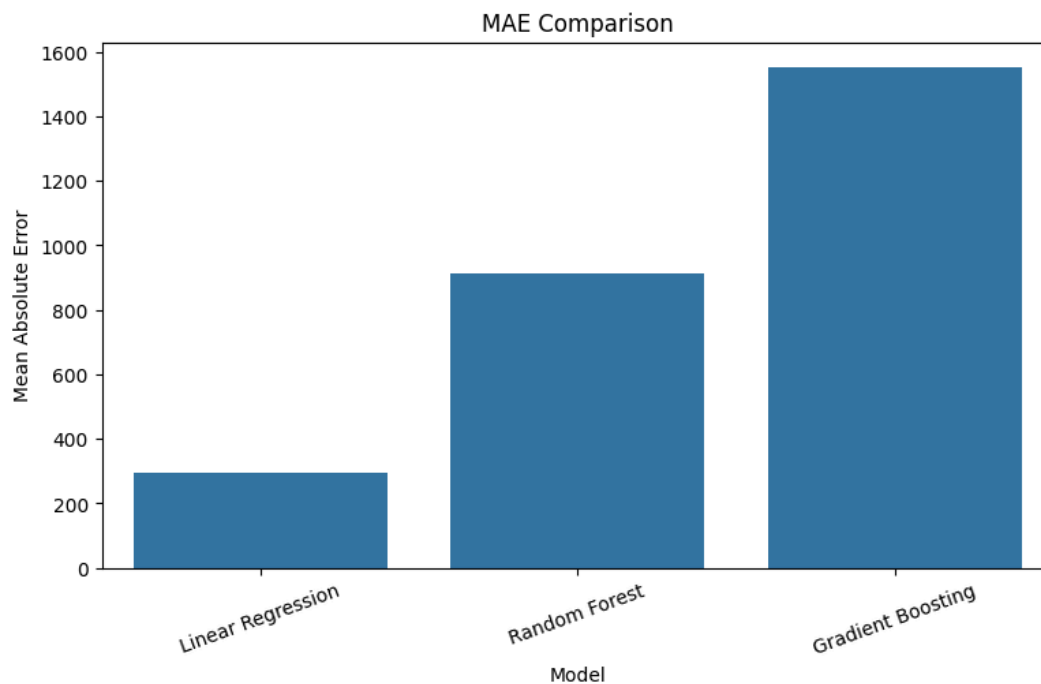
plt.xticks(rotation=20)

plt.show()

```



```
plt.figure(figsize=(9, 5))
sns.barplot(
    data=results_df,
    x="Model",
    y="MAE"
)
plt.title("MAE Comparison")
plt.xlabel("Model")
plt.ylabel("Mean Absolute Error")
plt.xticks(rotation=20)
plt.show()
```



```
def review_detection_system(review, rating=None, category=None):

    # Clean review
    cleaned_review = clean_text(review)

    # Convert text to TF-IDF
    review_vector = tfidf.transform([cleaned_review])

    # Predict label
    prediction = model.predict(review_vector)[0]

    # Prediction probabilities
    probabilities = model.predict_proba(review_vector)[0]

    # Find Fake probability
    fake_index = list(model.classes_).index("CG")
    genuine_index = list(model.classes_).index("OR")
```

```
fake_probability = probabilities[fake_index] * 100
genuine_probability = probabilities[genuine_index] * 100

# Final result
if prediction == "CG":
    result = "FAKE REVIEW"
else:
    result = "GENUINE REVIEW"

print("=" * 50)
print("          TRUSTREVIEW AI")
print("=" * 50)

print("\nReview:")
print(review)

if category is not None:
    print("\nCategory:", category)

if rating is not None:
    print("Rating:", rating)

print("\nPrediction:", result)
print(f"Fake Probability: {fake_probability:.2f}%")
print(f"Genuine Probability: {genuine_probability:.2f}%")

print("=" * 50)

return {
    "review": review,
    "prediction": result,
    "fake_probability": round(fake_probability, 2),
    "genuine_probability": round(genuine_probability, 2),
    "rating": rating,
    "category": category
}
```

```
results = []

for name, model in models.items():

    model.fit(X_train_views, y_train_views)

    predictions = model.predict(X_test_views)

    mae = mean_absolute_error(y_test_views, predictions)
    rmse = np.sqrt(mean_squared_error(y_test_views, predictions))
    r2 = r2_score(y_test_views, predictions)

    results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2 Score": r2
    })

results_df = pd.DataFrame(results)

print("Views Prediction Model Comparison:")
display(results_df)
```

Views Prediction Model Comparison:

	Model	MAE	RMSE	R2 Score
0	Linear Regression	293.837737	378.792333	-0.911178
1	Random Forest	911.185000	916.471501	-10.187597
2	Gradient Boosting	1551.835398	1556.260654	-31.259940

```
models = {
    "Linear Regression": LinearRegression(),

    "Random Forest": RandomForestRegressor(
        n_estimators=200,
        random_state=42
    ),

    "Gradient Boosting": GradientBoostingRegressor(
        n_estimators=200,
        random_state=42
    )
}
```

```
print("Models created successfully! ✅")
```

Models created successfully! ✅

```
def review_detection_system(review, rating=None, category=None):

    cleaned_review = clean_text(review)

    review_vector = tfidf.transform([cleaned_review])

    prediction = fake_model.predict(review_vector)[0]
    probabilities = fake_model.predict_proba(review_vector)[0]

    fake_index = list(fake_model.classes_).index("CG")
    genuine_index = list(fake_model.classes_).index("OR")

    fake_probability = probabilities[fake_index] * 100
    genuine_probability = probabilities[genuine_index] * 100

    if prediction == "CG":
        result = "FAKE REVIEW"
    else:
        result = "GENUINE REVIEW"

    print("=" * 55)
    print("                TRUSTREVIEW AI")
    print("=" * 55)

    print("\nReview:")
    print(review)

    if category is not None:
        print("\nCategory:", category)

    if rating is not None:
        print("Rating:", rating)

    print("\nPrediction:", result)
    print(f"Fake Probability: {fake_probability:.2f}%")
    print(f"Genuine Probability: {genuine_probability:.2f}%")

    print("=" * 55)

    return {
        "review": review,
        "prediction": result,
        "fake_probability": round(fake_probability, 2),
        "genuine_probability": round(genuine_probability, 2),
        "rating": rating,
        "category": category
    }
```

```
from sklearn.linear_model import LogisticRegression
```

```
fake_model = LogisticRegression(
    max_iter=1000,
    class_weight="balanced",
    random_state=42
)

fake_model.fit(
    X_train_tfidf,
    y_train
)

print("Fake/Genuine model trained successfully! ✅")
```

Fake/Genuine model trained successfully! ✅

```
print("Model classes:", fake_model.classes_)
print("Model ready for prediction! ✅")
```

Model classes: ['CG' 'OR']
Model ready for prediction! ✅

```
result = review_detection_system(
    review="""
    I purchased this product last week and I am
    very happy with the quality. It works exactly
    as described and the delivery was quick.
    """,
    rating=5,
    category="Electronics"
```

```

)

print("\nResult Dictionary:")
print(result)

=====
TRUSTREVIEW AI
=====

Review:

    I purchased this product last week and I am
    very happy with the quality. It works exactly
    as described and the delivery was quick.

Category: Electronics
Rating: 5

Prediction: GENUINE REVIEW
Fake Probability: 1.98%
Genuine Probability: 98.02%
=====

Result Dictionary:
{'review': '\n    I purchased this product last week and I am\n    very happy with the quality. It works exactly\n    as described and

```

```
!pip install -q transformers torch
```

```

from transformers import pipeline

sentiment_analyzer = pipeline(
    "sentiment-analysis",
    model="cardiffnlp/twitter-xlm-roberta-base-sentiment"
)

print("Multilingual Sentiment Analyzer loaded successfully! ✅")

```

```

config.json: 100%                               841/841 [00:00<00:00, 50.5kB/s]
pytorch_model.bin: reconstructing file: 100%          1.11GB / 1.11GB, 60.3MB/s
pytorch_model.bin: downloading bytes:                1.05GB, 59.7MB/s
model.safetensors: reconstructing file: 100%          1.11GB / 1.11GB, 67.5MB/s
model.safetensors: downloading bytes:                1.05GB, 34.4MB/s
Loading weights: 100%                             201/201 [00:00<00:00, 5176.66it/s]
sentencepiece.bpe.model: reconstructing file: 100%    5.07MB / 5.07MB, 403kB/s
sentencepiece.bpe.model: downloading bytes:          3.51MB, 203kB/s
special_tokens_map.json: 100%                     150/150 [00:00<00:00, 968B/s]
Multilingual Sentiment Analyzer loaded successfully! ✅

```

```

from transformers import pipeline

sentiment_analyzer = pipeline(
    "sentiment-analysis",
    model="cardiffnlp/twitter-xlm-roberta-base-sentiment"
)

print("Multilingual Sentiment Analyzer loaded successfully! ✅")

```

```

Loading weights: 100%                             201/201 [00:00<00:00, 2442.35it/s]
Multilingual Sentiment Analyzer loaded successfully! ✅

```

```

review = """
I purchased this product last week and I am
very happy with the quality. It works exactly
as described and the delivery was quick.
"""

sentiment_result = sentiment_analyzer(review)

print("Sentiment Result:")
print(sentiment_result)

Sentiment Result:
[{'label': 'positive', 'score': 0.8865218162536621}]

```

```

def trustreview_ai(review, rating=None, category=None):

    # Clean review

```


I absolutely love this product! The quality is amazing,
delivery was fast, and everything works perfectly.
I would definitely recommend it to everyone.

Category: Electronics
Rating: 5

--- Review Analysis ---
Prediction: GENUINE REVIEW
Fake Probability: 9.63%
Genuine Probability: 90.37%

--- Sentiment Analysis ---
Sentiment: positive
Sentiment Confidence: 93.25%

=====

Final Result:
{'review': '\n I absolutely love this product! The quality is amazing,\n delivery was fast, and everything works perfectly.\n

```
def product_analytics(category):  
  
    product = product_stats[  
        product_stats["category"].str.lower() == category.lower()  
    ]  
  
    if product.empty:  
        print("Category not found ❌")  
        print("\nAvailable categories:")  
        print(product_stats["category"].head(20).tolist())  
        return None  
  
    product = product.iloc[0]  
  
    print("=" * 60)  
    print("          PRODUCT ANALYTICS")  
    print("=" * 60)  
  
    print("\nCategory:", product["category"])  
    print("Total Reviews:", int(product["total_reviews"]))  
    print("Average Rating:", round(product["average_rating"], 2))  
    print("Fake Reviews:", int(product["fake_reviews"]))  
    print("Genuine Reviews:", int(product["genuine_reviews"]))  
    print("Fake Percentage:", round(product["fake_percentage"], 2), "%")  
    print("Genuine Percentage:", round(product["genuine_percentage"], 2), "%")  
    print("Estimated Views:", int(product["estimated_views"]))  
  
    print("=" * 60)  
  
    return product
```

```
# First category from our dataset  
test_category = product_stats["category"].iloc[0]  
  
product_analytics(test_category)
```

=====

PRODUCT ANALYTICS

=====

Category: Books_5
Total Reviews: 3489
Average Rating: 4.28
Fake Reviews: 1754
Genuine Reviews: 1735
Fake Percentage: 50.27 %
Genuine Percentage: 49.73 %
Estimated Views: 112279

=====

0	
category	Books_5
total_reviews	3489
average_rating	4.28
fake_reviews	1754
genuine_reviews	1735
fake_percentage	50.272284
genuine_percentage	49.727716
estimated_views	112279

dtype: object

```

def final_trustreview_ai(review, rating=None, category=None):

    # =====
    # 1. REVIEW PREPROCESSING
    # =====
    cleaned_review = clean_text(review)

    # =====
    # 2. FAKE / GENUINE DETECTION
    # =====
    review_vector = tfidf.transform([cleaned_review])

    prediction = fake_model.predict(review_vector)[0]
    probabilities = fake_model.predict_proba(review_vector)[0]

    fake_index = list(fake_model.classes_).index("CG")
    genuine_index = list(fake_model.classes_).index("OR")

    fake_probability = probabilities[fake_index] * 100
    genuine_probability = probabilities[genuine_index] * 100

    if prediction == "CG":
        review_result = "FAKE REVIEW"
    else:
        review_result = "GENUINE REVIEW"

    # =====
    # 3. SENTIMENT ANALYSIS
    # =====
    sentiment_result = sentiment_analyzer(review)[0]

    sentiment_label = sentiment_result["label"]
    sentiment_confidence = sentiment_result["score"] * 100

    # =====
    # 4. DISPLAY REVIEW ANALYSIS
    # =====
    print("=" * 65)
    print("                                TRUSTREVIEW AI")
    print("=" * 65)

    print("\n 📄 REVIEW")
    print("-" * 65)
    print(review)

    if rating is not None:
        print("\n ⭐ Rating:", rating)

    if category is not None:
        print("📦 Category:", category)

    print("\n 🔍 REVIEW AUTHENTICITY")
    print("-" * 65)
    print("Prediction:", review_result)
    print(f"Fake Probability: {fake_probability:.2f}%")
    print(f"Genuine Probability: {genuine_probability:.2f}%")

    print("\n 😊 SENTIMENT ANALYSIS")
    print("-" * 65)
    print("Sentiment:", sentiment_label)
    print(f"Confidence: {sentiment_confidence:.2f}%")

    # =====
    # 5. PRODUCT ANALYTICS
    # =====
    product_data = None

    if category is not None:

        product = product_stats[
            product_stats["category"].str.lower() == category.lower()
        ]

        if not product.empty:

            product = product.iloc[0]
            product_data = product

            print("\n 📊 PRODUCT ANALYTICS")
            print("-" * 65)
            print("Total Reviews:", int(product["total_reviews"]))
            print("Average Rating:", round(product["average_rating"], 2))
            print("Fake Reviews:", int(product["fake_reviews"]))
            print("Genuine Reviews:", int(product["genuine_reviews"]))
            print("Fake Percentage:", round(product["fake_percentage"], 2), "%")
            print("Genuine Percentage:", round(product["genuine_percentage"], 2), "%")

```

```

        print("Estimated Views:", int(product["estimated_views"]))

    else:
        print("\n🚨 Category not found in product analytics.")

print("\n" + "=" * 65)
print("                ANALYSIS COMPLETED ✅")
print("=" * 65)

return {
    "review": review,
    "prediction": review_result,
    "fake_probability": round(fake_probability, 2),
    "genuine_probability": round(genuine_probability, 2),
    "sentiment": sentiment_label,
    "sentiment_confidence": round(sentiment_confidence, 2),
    "rating": rating,
    "category": category
}

```

```

final_result = final_trustreview_ai(
    review="""
    This product is absolutely amazing! The quality is excellent,
    delivery was very fast, and I am extremely satisfied.
    I would definitely recommend this product to everyone.
    """,
    rating=5,
    category=product_stats["category"].iloc[0]
)

print("\nFINAL RESULT DICTIONARY:")
print(final_result)

```

```

=====
                        TRUSTREVIEW AI
=====

📄 REVIEW
-----

    This product is absolutely amazing! The quality is excellent,
    delivery was very fast, and I am extremely satisfied.
    I would definitely recommend this product to everyone.

    ★ Rating: 5
    📦 Category: Books_5

🔍 REVIEW AUTHENTICITY
-----
Prediction: GENUINE REVIEW
Fake Probability: 1.02%
Genuine Probability: 98.98%

😊 SENTIMENT ANALYSIS
-----
Sentiment: positive
Confidence: 92.65%

📊 PRODUCT ANALYTICS
-----
Total Reviews: 3489
Average Rating: 4.28
Fake Reviews: 1754
Genuine Reviews: 1735
Fake Percentage: 50.27 %
Genuine Percentage: 49.73 %
Estimated Views: 112279

=====
                        ANALYSIS COMPLETED ✅
=====

FINAL RESULT DICTIONARY:
{'review': '\n    This product is absolutely amazing! The quality is excellent,\n    delivery was very fast, and I am extremely satisfi

```

```

import joblib
import os

# Create folder
os.makedirs("trustreview_models", exist_ok=True)

# Save Fake/Genuine model
joblib.dump(
    fake_model,
    "trustreview_models/fake_review_model.pkl"
)

```

```
# Save TF-IDF vectorizer
joblib.dump(
    tfidf,
    "trustreview_models/tfidf_vectorizer.pkl"
)

# Save Views prediction model
joblib.dump(
    rf_views,
    "trustreview_models/views_model.pkl"
)

# Save product analytics data
product_stats.to_csv(
    "trustreview_models/product_stats.csv",
    index=False
)

print("All TrustReview AI files saved successfully! 🟢")
```

All TrustReview AI files saved successfully! 🟢